PYTHON OOP ASSIGNMENT

Shopping Cart System

Objective

Create a simple Shopping Cart System using only two Python classes: **Product** and **ShoppingCart**. You will practise constructors, private attributes, methods, object interaction, and basic calculations.

1. Problem Description

Imagine you are developing a simple online shopping application. A customer can view product details, add products to a shopping cart, update product quantities, remove products, calculate the total bill, and apply a discount.

Your task is to implement these functionalities using Python Object-Oriented Programming (OOP).

2. Learning Objectives

- Create classes and objects.
- Understand constructors (__init__).
- Implement encapsulation using private attributes.
- Create methods to manage products.
- Work with lists and dictionaries inside classes.

3. Task 1: Create a Product Class

Create a class named **Product**. This class represents an individual product available in the store.

Required private attributes:

Attribute	Description	Example
__name	Product name	Laptop
__price	Product price	100000
__quantity	Available quantity	10

Requirements:

1. Create a constructor to initialize all three attributes.
2. Make all attributes private using double underscores.
3. Create a display_product() method to display product details.
4. Create getter methods to access product name, price, and quantity.

Example:

```
product1 = Product("Laptop", 100000, 10)\nproduct2 = Product("Mouse", 1500, 20)\nproduct3 =
Product("Keyboard", 2500, 15)
```

Expected output:

```
Product: Laptop\nPrice: Rs. 100000\nAvailable Quantity: 10
```

4. Task 2: Create a ShoppingCart Class

Create another class named **ShoppingCart**.

Required private attribute:

- **__cart_items:** A list to store products added to the cart.

Requirements:

1. Create a constructor to initialize an empty list.
2. Create a method named add_product(product, quantity).
3. Allow customers to add product objects to the cart.
4. Store the selected quantity of each product.
5. Do not allow customers to add more items than available stock.

Hint: You can store product objects and their quantities using dictionaries.

```
cart_items = [\n    {"product": product1, "quantity": 1},\n    {"product": product2, "quantity": 2}\n]
```

5. Task 3: Update Product Quantity

Create a method:

```
update_quantity(product_name, new_quantity)
```

Requirements:

1. Find the product in the shopping cart.
2. Update its quantity.
3. Check that the new quantity is greater than zero.
4. Make sure the requested quantity does not exceed available stock.
5. Display a success or error message.

Example:

```
Product: Mouse\nOld Quantity: 2\nNew Quantity: 5\n\nQuantity updated successfully!
```

6. Task 4: Remove Product from Cart

Create a method:

```
remove_product(product_name)
```

Requirements:

1. Search for the product in the cart.
2. Remove it if it exists.
3. Display a confirmation message.
4. Display an error message if the product is not found.

7. Task 5: Display Shopping Cart

Create a method:

```
display_cart()
```

Display all products currently added to the shopping cart.

```
========== SHOPPING CART ==========\n\nProduct       Price       Quantity\n----------------------
-----------\nLaptop        100000         1\nMouse           1500         2\nKeyboard        2500
1\n\n==================================
```

8. Task 6: Calculate Total Bill

Create a method:

```
calculate_total()
```

Calculate the total amount of all products in the shopping cart.

Formula: Subtotal = Product Price × Quantity. Cart Total = Sum of all product subtotals.

Product	Price (Rs.)	Quantity	Subtotal (Rs.)
Laptop	100,000	1	100,000
Mouse	1,500	2	3,000
Keyboard	2,500	1	2,500
TOTAL			105,500

Expected output:

```
Total Shopping Amount: Rs. 105500
```

9. Task 7: Apply Discount

Create a method:

```
apply_discount(discount_percentage)
```

Requirements:

5. Accept a discount percentage.
6. Calculate the discount amount.
7. Subtract the discount from the total bill.
8. Display the original amount, discount, and final amount.

Example: Original amount = Rs. 105,500; Discount = 10%.

```
Discount Amount = 105500 × 10 / 100 = Rs. 10550\nFinal Amount = 105500 - 10550 = Rs. 94950
```

Expected output:

```
Original Amount: Rs. 105500\nDiscount: 10%\nDiscount Amount: Rs. 10550\n\nFinal Amount: Rs. 94950
```

10. Sample Product Data

Create the following product objects:

Product	Price (Rs.)	Available Quantity
Laptop	100,000	10
Mouse	1,500	20
Keyboard	2,500	15
Headphones	3,500	12
USB Cable	500	30

11. Develop a Menu-Driven Application

Once you have implemented all the required methods, create a menu-driven application.

```
========== SHOPPING CART SYSTEM ==========\n\n1. Display Products\n2. Add Product to Cart\n3. Update Product Quantity\n4. Remove Product\n5. Display Shopping Cart\n6. Calculate Total\n7. Apply Discount\n8. Exit\n\nEnter your choice:
```

Program requirements:

- Use input() to accept customer choices.
- Use if-elif-else for menu operations.
- Use a while loop to keep the application running.
- Use a list to store available products.
- Use objects to represent products.
- Use ShoppingCart methods to manage shopping operations.
- Validate product IDs or names and quantities.

12. Expected Program Execution

```
========== SHOPPING CART SYSTEM ==========\n\n1. Display Products\n2. Add Product to Cart\n3. Update Product Quantity\n4. Remove Product\n5. Display Shopping Cart\n6. Calculate Total\n7. Apply Discount\n8. Exit\n\nEnter your choice: 2\nEnter Product Name: Mouse\nEnter Quantity: 2\nMouse added successfully!\n\nEnter your choice: 6\nTotal Shopping Amount: Rs. 3000\n\nEnter your choice: 7\nEnter Discount Percentage: 10\nDiscount Amount: Rs. 300\nFinal Amount: Rs. 2700
```

Complete each task step by step. First create the Product class, then create the ShoppingCart class. Understand how objects of one class can be passed to methods of another class.